

FIAP | DSA

Aula 2: Fundamentos de Organização de Dados

Structs, Ponteiros e Typedef

Prof. Dr. Erick Toshio Yamamoto

Objetivos da Aula



Compreender a necessidade de criar tipos de dados complexos em C.



Dominar a sintaxe de declaração e uso de Structs.



Aprender a simplificar o código usando o Typedef.



Estabelecer as bases teóricas de Ponteiros para estruturas lineares.

Por que tipos personalizados?

Tipos Primitivos Isolados

Variáveis como `int`, `float` e `char` são ótimas para dados únicos, mas limitadas para objetos complexos.

Como representar um **Aluno** usando apenas tipos primitivos?

int rm; char nome[50]; float nota;

⚠ **Problema: Os dados ficam "soltos" e sem conexão lógica no código.**



Entidade: Aluno

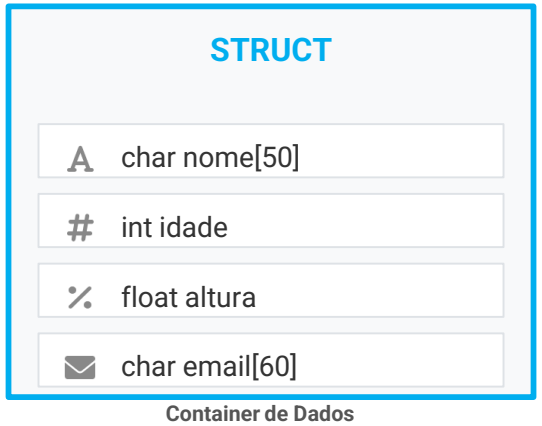
Uma struct agrupa dados relacionados em uma única unidade lógica.

- RM (int)
- Nome (char[])
- Nota (float)
- E-mail (char[])

O Conceito de Struct

Uma Struct (Estrutura) é um tipo de dado definido pelo usuário que permite agrupar variáveis de tipos diferentes.

- **Heterogeneidade:** Ao contrário dos arrays, pode conter tipos distintos (int, float, char).
- **Unidade Lógica:** Trata um conjunto de dados como uma única entidade.
- **Organização:** Facilita a passagem de grandes volumes de dados para funções.



Container de Dados

Sintaxe da Struct em C

Palavra-chave `struct`

Indica ao compilador que estamos definindo um novo tipo de estrutura.

Tag (Nome)

O identificador da estrutura (ex: Aluno, Carro, Produto).

Membros

Variáveis internas de qualquer tipo, declaradas dentro das chaves.

ATENÇÃO: Nunca esqueça o ponto e vírgula (;) após fechar a chave da struct!

```c
// Definição da estrutura
struct Aluno {
    int rm;
    char nome[50];
    float nota;
}; // <--- Ponto e vírgula obrigatório

// Declaração de uma variável do tipo struct
struct Aluno aluno1;
```

Acesso aos Membros: Operador Ponto

O OPERADOR PONTO

Para acessar qualquer campo de uma variável do tipo struct, utilizamos o caractere ponto (.).

Sintaxe:

```
variavel.membro
```

Funciona tanto para atribuição quanto para leitura de valores.

```c
// 1. Atribuição (Escrita)
aluno1.rm = 12345;
aluno1.nota = 9.5;
strcpy(aluno1.nome, "Joao Silva");

// 2. Acesso (Leitura)
printf("RM: %d\n", aluno1.rm);
printf("Nota: %.1f\n", aluno1.nota);

// 3. Uso em Expressões
float bonus = aluno1.nota + 0.5;
```

Memória e Structs

Armazenamento Contíguo

Os membros de uma struct são armazenados em endereços de memória sequenciais, na ordem em que foram declarados.

Alinhamento (Alignment)

Processadores acessam a memória de forma mais eficiente em blocos (ex: 4 ou 8 bytes). O compilador ajusta os dados para esses limites.

Padding (Preenchimento)

Bytes vazios inseridos pelo compilador entre os membros para garantir o alinhamento correto. Isso pode aumentar o tamanho total da struct.

Exemplo: struct { char c; int i; }



1 Byte 4 Bytes Padding

O tamanho total será 8 bytes, e não 5, devido ao alinhamento de 4 bytes do `int` .

Introdução ao Typedef

O QUE É?

O comando `typedef` é utilizado para criar um **apelido** (alias) para um tipo de dado já existente.

POR QUE USAR?

Melhora a legibilidade do código, facilita a manutenção e reduz a verbosidade, especialmente ao lidar com structs.

Pense no `typedef` como uma forma de dar nomes mais significativos aos seus dados.

```c
// Sintaxe: typedef tipo_original novo_nome;

typedef unsigned long ulong;
typedef int inteiro;


// Agora podemos usar os novos nomes:
ulong populacao = 8000000000;
inteiro idade = 25;


/* O compilador trata 'ulong' exatamente como 'unsigned long'. */
```

A Forma Profissional: Struct + Typedef

SINTAXE SIMPLIFICADA

Ao combinar `typedef` com `struct`, eliminamos a necessidade de repetir a palavra-chave `struct` em cada declaração de variável.

Antes:

```
struct Aluno a1;
```

Depois (com Typedef):

```
Aluno a1;
```

Este é o padrão utilizado na maioria das bibliotecas e projetos profissionais em C.

```c
// Definição unificada
typedef struct {
    int rm;
    char nome[50];
    float nota;
} Aluno;


// Agora a declaração é muito mais limpa:
Aluno a1, a2;

// Uso normal com o operador ponto:
a1.rm = 98765;
a1.nota = 10.0;
```

Structs Aninhadas

COMPOSIÇÃO

Ocorre quando um dos membros de uma struct é, por si só, uma outra struct previamente definida.

HIERARQUIA

Permite modelar dados complexos de forma organizada e modular, refletindo melhor as entidades do mundo real.

Acesso Encadeado:

`var.membro_struct.campo`

```c
typedef struct
{
   char rua[50];
   int numero;
} Endereco;

typedef struct {
   char nome[50];
   Endereco end; // Struct aninhada
} Cliente;

// Uso no código:
Cliente c1;
strcpy(c1.nome, "Ana");
c1.end.numero = 100; // Acesso duplo
```

Atividade em Sala: Criando o Registro Produto

O Desafio

Escreva um programa em C que defina e utilize uma estrutura para gerenciar produtos de um estoque.

- Use `typedef struct` para definir **Produto**.
- Campos: ID (int), Nome (char[50]), Preço (float).
- Leia os dados de 1 produto via teclado.
- Exiba os dados formatados na tela.

```c
// Esqueleto da Atividade
typedef struct {
    int id;
    char nome[50];
    float preco;
} Produto;

int main() {
    Produto p1;
    // Implemente a leitura e exibição aqui
    return 0;
}
```

Resumo da Parte 1: Organização de Dados



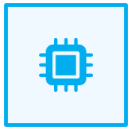
STRUCTS

Permitem agrupar variáveis de tipos diferentes em uma única unidade lógica, representando entidades do mundo real.



ACESSO

O **operador ponto (.)** é a chave para ler e escrever nos membros de uma variável do tipo estrutura.



MEMÓRIA

Structs ocupam blocos contíguos na RAM, onde o **alinhamento** e o **padding** garantem a eficiência da CPU.



TYPEDEF

Cria apelidos para tipos complexos, tornando o código mais limpo, legível e fácil de manter profissionalmente.

Próximo: Ponteiros e Aplicações →

Ponteiros e Aplicações em Structs

Endereçamento e Manipulação de Memória

Aula 2 | Parte 2 | FIAP

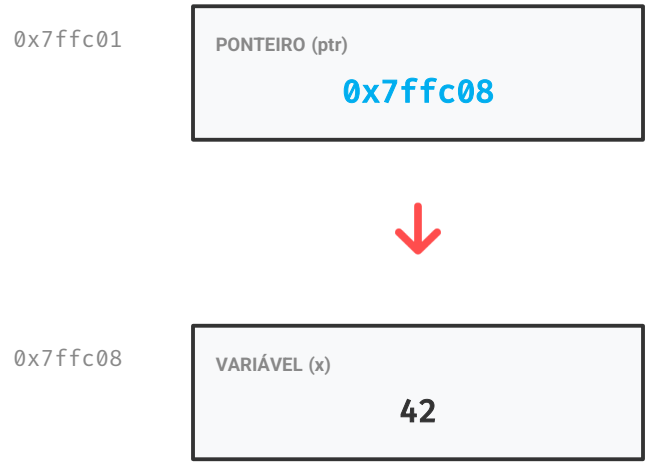


DSA | FIAP

O que são Ponteiros?

Um ponteiro é uma variável cujo valor é o endereço de memória de outra variável.

- **Endereçamento:** Não armazena o dado, mas sim "onde" o dado está na RAM.
- **Referência:** Permite manipular variáveis de forma indireta.
- **Eficiência:** Evita a cópia de grandes blocos de dados entre funções.



Operadores de Ponteiros: & e *



ENDEREÇO DE

Retorna o endereço de memória de uma variável existente.

```
int x = 10;
int* ptr = &x;
// ptr guarda o local de x
```



DESREFERÊNCIA

Acessa o valor armazenado no endereço que o ponteiro aponta.

```
int valor = *ptr;
*ptr = 20;
// altera x via ponteiro
```

Por que usar Ponteiros com Structs?

O Custo da Cópia

Ao passar uma struct por valor para uma função, o C cria uma cópia completa de todos os seus membros.

- Consumo excessivo de memória RAM.
- Perda de performance (overhead de CPU).
- Alterações na cópia não afetam a original.

Eficiência do Endereço

Ao usar ponteiros, passamos apenas o endereço de memória (4 ou 8 bytes), independente do tamanho da struct.

- Acesso direto e instantâneo aos dados.
- Economia drástica de recursos do sistema.
- Permite que funções modifiquem a struct original.

O Operador Seta (->)

O QUE É?

É um operador de conveniência usado para acessar membros de uma struct através de um ponteiro.

->

EQUIVALÊNCIA

A expressão `ptr->membro` é exatamente igual a `(*ptr).membro`.

💡*Facilita a leitura e evita o uso excessivo de parênteses.*

```c
// Supondo a struct Aluno e um ponteiro
Aluno a1 = {123, "Ana", 9.0};
Aluno* ptr = &a1;

// Forma 1: Desreferência + Ponto
(*ptr).nota = 10.0;

// Forma 2: Operador Seta (Recomendado)
ptr->nota = 10.0;

// Exibindo dados
printf("Nome: %s", ptr->nome);
```

Passagem por Valor vs. Referência

POR VALOR (CÓPIA)

- Uma cópia completa dos dados é criada na pilha.
- Ineficiente para structs grandes (gasta RAM e CPU).
- Seguro: a função não altera a variável original.

```
void func(Aluno a) {
   a.nota = 10;
  // Altera apenas a cópia local
}
```

POR REFERÊNCIA (ENDEREÇO)

- Apenas o endereço (8 bytes) é passado.
- Extremamente rápido, independente do tamanho da struct.
- Permite que a função modifique o dado original.

```
void func(Aluno* a) {
   a->nota = 10;
  // Altera o original via ponteiro
}
```

Funções e Ponteiros de Structs

POR QUE PASSAR O ENDEREÇO?

Ao passar uma struct por valor, o C cria uma cópia completa. Para structs grandes, isso desperdiça tempo e memória.

ALTERAÇÃO DIRETA

Passar um ponteiro permite que a função altere os dados originais da struct, sem precisar retornar uma nova estrutura.

🛠 Dentro da função, usamos o operador `->` para acessar os membros.

```c
// Função que altera o dado original
void atualizarNota(Aluno* p, float nova) {
    if (nova >= 0 && nova <= 10) {
        p->nota = nova; // Altera o original
    }
}

// Chamada na main
int main() {
    Aluno a1 = {123, "Ana", 8.0};

    // Passamos o endereço com &
    atualizarNota(&a1, 9.5);

    printf("Nova nota: %.1f", a1.nota);
    return 0;
}
```

Vetores de Structs e Ponteiros

O NOME DO VETOR

Em C, o nome de um vetor funciona como um ponteiro constante para o seu primeiro elemento (índice 0).

ARITMÉTICA DE PONTEIROS

Ao somar 1 a um ponteiro de struct, o C avança exatamente o número de bytes que a struct ocupa na memória.

```
vet[i] == *(vet + i)
```

```c
// Vetor de 3 alunos
Aluno turma[3];

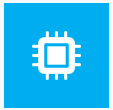
// Ponteiro apontando para o início
Aluno* ptr = turma;

// Acessando o primeiro aluno
ptr->rm = 101;

// Avançando para o próximo (Aritmética)
ptr++; // Agora aponta para turma[1]
ptr->rm = 102;

// Percorrendo com loop
for(int i = 0; i < 3; i++) {
   printf("%d\n", (turma + i)->rm);
}
```

Aplicações Reais: Onde usamos isso?



SISTEMAS OPERACIONAIS

O kernel do Linux usa structs para representar processos (PCB), arquivos e drivers de hardware, manipulados via ponteiros para máxima velocidade.



GAME ENGINES

Motores como Unreal e Unity (C++) usam estruturas para gerenciar entidades, coordenadas 3D e estados de jogo em tempo real.



SISTEMAS EMBARCADOS

Em IoT e automação, ponteiros acessam diretamente endereços de memória de sensores e atuadores, economizando cada byte de RAM.



ESTRUTURAS DE DADOS

A base para Listas Ligadas, Árvores e Grafos. Sem ponteiros e structs, seria impossível criar estruturas dinâmicas e flexíveis.

Boas Práticas com Ponteiros

Inicialização Segura

Sempre inicialize ponteiros com `NULL` se não tiver um endereço imediato. Isso evita que o ponteiro aponte para "lixo" na memória.

Verificação de Validade

Antes de desreferenciar (usar o `*` ou `->`), verifique se o ponteiro não é nulo: `if (ptr != NULL)`.

Evite Ponteiros "Soltos" (Dangling Pointers)

Após usar `free()` em um ponteiro, atribua `NULL` a ele imediatamente para evitar acessos a memória já liberada.

Documentação e Clareza

Use nomes significativos (ex: `p_aluno` em vez de `p`) e comente quem é o "dono" da memória apontada.

Atividade em Sala: Aplicando Desconto via Ponteiro

O Desafio

Crie uma função que receba o endereço de um **Produto** e aplique uma porcentagem de desconto no seu preço.

- [x] Protótipo: `void aplicarDesconto(Produto* p, float desc)` .
- [x] Use o operador `->` para alterar o preço original.
- [x] Na `main`, passe o endereço da variável usando `&`.
- [x] Exiba o preço antes e depois da chamada.

```c
// Esqueleto da Atividade
void aplicarDesconto(Produto* p, float desc) {

    // Calcule e atualize o preço aqui
    p->preco = ...
}

int main() {
    Produto prod = {1, "Teclado", 150.0};

    // Chame a função passando o endereço
    aplicarDesconto(&prod, 10.0);
    return 0;
}
```

Desafio para Casa e Próxima Aula

Desafio Prático

Crie um mini-sistema de biblioteca que utilize uma struct **Livro**.

- Use `typedef struct` para os dados do livro.
- Crie uma função que receba um **ponteiro** para o livro.
- A função deve alterar o status de "disponível" para "emprestado".

Próxima Aula

Na Aula 4, daremos o passo final para as estruturas dinâmicas:

Alocação Dinâmica (malloc/free)

Início das Listas Ligadas

Preparem-se para gerenciar a memória RAM manualmente!